Parallel & Distributed Computing

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Submitted By:-
Jayant Batra
102303825
3C61

Submitted To:-
Dr. Saif Nalbad

Matrix Multiplication – Parallel Implementation and Analysis

Q2. In this question, I had to build a parallel implementation of matrix multiplication for large matrices (1000×1000) using OpenMP. The goal was similar to Question 1, but this time the workload is much heavier and more compute-intensive.

I implemented **two different parallelization strategies**:

1. **1D threading** – only one loop is parallelized
2. **2D threading** – nested loops are parallelized using OpenMP `collapse(2)`

I then repeated the same kind of analysis as in Q1:

- execution time
- scaling with threads
- and finally perf stat to understand what is actually happening inside the CPU.

How I implemented matrix multiplication

The basic formula is:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \times B[k][j]$$

Matrix size was fixed at 1000×1000, so the total work is around 10^9 multiply-add operations, which is much larger than DAXPY.

All matrices were stored in 1D vectors in row-major order.

Step 1: Simple 1D Parallel Version

In the first version, I parallelized only the **outermost loop (i loop)**.

```
#pragma omp parallel for num_threads(threads)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        double sum = 0.0;
        for (int k = 0; k < N; k++) {
            sum += A[i*N + k] * B[k*N + j];
        }
        C[i*N + j] = sum;
    }
}
```

Step 2: 1D Version with Transposed B

While analyzing memory access, I realized that `B[k][j]` causes **strided memory access**, which hurts cache performance.

To fix this, I explicitly **transposed matrix B** before multiplication.

```
sum += A[i*N + k] * B_trans[j*N + k];
```

Step 3: 2D Parallelization using `collapse(2)`

Next, I parallelized both `i` and `j` loops:

```
#pragma omp parallel for num_threads(threads) collapse(2)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        ...
    }
}
```

Step 4: 2D + Transpose (My Best Version)

Finally, I combined:

1) 2D parallelization

2) transposed B

This version has:

1) best cache locality
2) best load balancing
3) minimum idle or wait time at high thread counts

Execution Time Results

Threads	Simple 1D	Transpose 1D	2D Collapse	2D + Transpose
1	8.55	8.51	8.74	**7.86**
2	4.68	4.58	4.56	**4.25**
4	2.66	2.58	2.49	**2.48**
8	1.48	1.51	1.48	**1.42**
16	1.21	**1.09**	1.15	**1.09**
32	1.25	1.11	1.11	**0.96**

The **2D transpose version** consistently performed the best, especially at higher thread counts.

Speedup Behavior

Using the best single-thread time as baseline:

- 8 threads → ~5.5× speedup
- 16 threads → ~7.2× speedup
- 32 threads → ~8.1× speedup

Unlike DAXPY, the performance **did not collapse immediately** after 8 threads.

1. Task Clock vs Wall Time

As threads increase:

- wall-clock time decreases
- task clock increases

This means the CPU is doing more total work to reduce execution time.

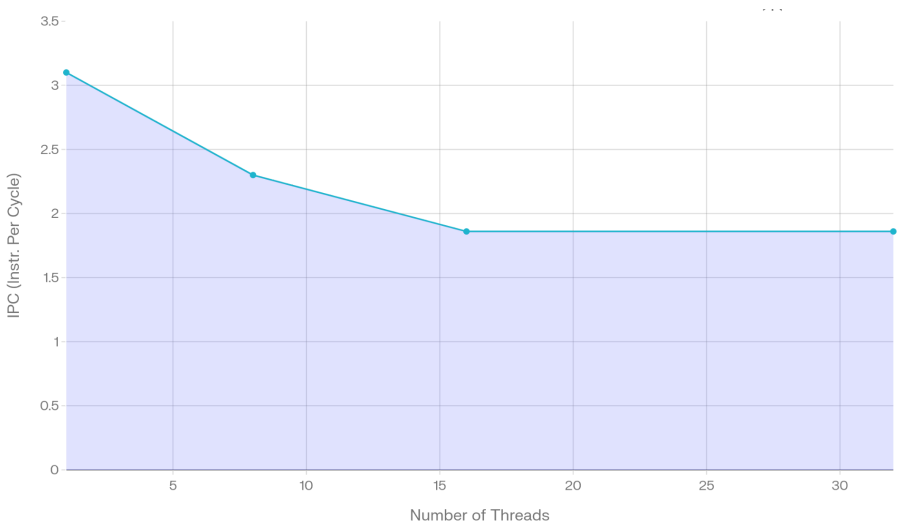
Example:

- 8 threads → ~42 sec task clock, ~6 sec wall time
- 16 threads → ~62 sec task clock, ~4.6 sec wall time

This shows diminishing efficiency at higher thread counts.

2. IPC Degradation

Threads	IPC
1	~3.1
8	~2.3
16	~1.86
32	~1.86



IPC Declining as Thread Count Increases, Higher thread counts lead to resource contention

IPC drops as threads increase, which means:

- more cycles are spent waiting
- mainly due to cache and memory pressure

But the drop is **much less severe than in DAXPY**.

Why Matrix Multiplication Scales Better than DAXPY

In Q1, DAXPY had very low arithmetic intensity around 0.08 FLOPS/bype

Matrix multiplication, on the other hand:

1) reuses data heavily
2) performs many computations per memory access

Why 2D + Transpose Works Best

From my observations:

1. Transposing B fixes poor cache access
2. collapse(2) improves load balancing
3. Threads finish at similar times
4. Less idle time and better CPU utilization

This combination consistently gave the **lowest execution time**.

Threads: Beginning of Resource Pressure

At 8 threads:

- Best execution time ≈ **1.42 s**
- CPU utilization ≈ **6.98**
- IPC ≈ **2.27**
- Frontend stalls increased to **2.16%**

This is a critical point.

The program is still scaling well, but I can clearly see:

- IPC steadily decreasing
- Task clock increasing significantly (~42s)
- Memory and cache pressure starting to appear

However, unlike DAXPY, performance does **not collapse** here.
Matrix multiplication has enough computation per memory access to keep cores busy.

Final Conclusion

This experiment clearly shows that not all parallel programs behave the same.

- DAXPY is memory-bound, so scaling breaks early
- Matrix multiplication is compute-heavy, so scaling continues further

However, even here:

- IPC drops
- task clock increases
- and efficiency falls beyond hardware limits

The best implementation on my system was 2D parallelization with transposed B, and the optimal thread range was 16–32 threads.

APPENDIX

System Architecture: AMD Ryzen 9 7940HS

The test system uses an AMD Ryzen 9 7940HS processor with the following specifications:

- Architecture: UMA (Uniform Memory Access) - single socket, non-NUMA
- Cores/Threads: 8 cores, 16 threads
- L3 Cache: 16 MB shared across all cores
- Memory: Single-channel DDR5-4800, 16 GB (theoretical peak bandwidth ~38.4 GB/s)
- Practical sustained bandwidth: Typically 40-60% of theoretical peak (~15-23 GB/s) for memory-intensive workloads